



MusicFlow

Ad-Free YouTube Music Player

Chrome Extension + Python Backend

Version 7.0.0

github.com/Sumitboii/musicflow

MusicFlow — Ad-Free YouTube Music Player



VERSION 7.0.0

PLATFORM CHROME EXTENSION

BACKEND PYTHON FLASK

AUDIO YT-DLP

QUOTA ZERO API COST

A Spotify-style Chrome extension that streams YouTube music as audio-only — no ads, no login, no API quota limits, no YouTube tab ever opens.

Table of Contents

- [What is MusicFlow?](#)
 - [How It Works](#)
 - [Features](#)
 - [Architecture](#)
 - [Tech Stack](#)
 - [Setup](#)
 - [Usage](#)
 - [Keyboard Shortcuts](#)
 - [File Structure](#)
 - [Why No API Quota?](#)
 - [Screenshots](#)
-

What is MusicFlow?

MusicFlow is a **Chrome browser extension** that turns YouTube into a pure music player — think Spotify, but powered by YouTube's massive library.

The problem with YouTube for music: -  Ads every few minutes -  Video loads even when you just want audio (wastes bandwidth) -  You need a browser tab open and visible -  YouTube Premium costs money

MusicFlow solves all of this: -  **Zero ads** — audio is extracted directly, bypasses the ad system -  **Audio only** — video never loads, saves 60-80% bandwidth -  **No visible tab** — plays silently in the background via an offscreen document -  **Completely free** — no subscription, no login, no account needed

How It Works

MusicFlow has two parts working together:

1. Chrome Extension (Frontend)

A Manifest V3 Chrome extension with: - **Popup UI** — Spotify-dark themed player with search, controls, playlists, lyrics - **Background Service Worker** — orchestrates messages between popup and player - **Offscreen Document** — a hidden HTML page with a `<audio>` element that plays the stream. The user never sees this page, but audio comes through the system speakers normally.

2. Python Backend (Local Server)

A lightweight Flask server running on `localhost:7842` that uses **yt-dlp** to: - Search YouTube (no API key needed) - Extract direct audio stream URLs from YouTube videos - Fetch YouTube Radio mixes for "Up Next" song suggestions - Provide search autocomplete suggestions

The Flow

```

User types "Arijit Singh"
    ↓
Extension → Backend /search → yt-dlp searches YouTube → returns 20 results
    ↓
User clicks a song
    ↓
Extension → Backend /stream → yt-dlp extracts audio URL → returns direct m4a URL
    ↓
Extension → Offscreen <audio src="...googlevideo.com/..."> → plays audio
    ↓
Windows Media Session API shows song name in system overlay
  
```

Everything happens locally on your machine. No data is sent to any third-party server.

Features

Search

- Powered by **yt-dlp** — unlimited searches, zero API quota cost
- Smart search suggestions as you type (debounced, uses Google autocomplete — also zero quota)
- Results show thumbnail, title, channel, and duration
- Search results cached for 30 minutes — same query costs nothing on repeat

Audio Playback

- **Audio-only streaming** — yt-dlp extracts the lowest-quality video stream (240p) that still has full-quality audio, saving significant bandwidth
- Plays via a native HTML5 `<audio>` element in a hidden offscreen document
- No YouTube tab, no visible window, no browser tab clutter
- Stream URLs cached for ~6 hours — replaying same song is instant

Player Controls

- Play / Pause
- Next / Previous song
- Seek bar with live time display
- Shuffle mode — random pick from queue
- Repeat mode — loop current song
- Volume slider with $\frac{1}{4}$ · $\frac{1}{2}$ · $\frac{3}{4}$ · Full presets
- Mute toggle

Playlists

- Save any search results queue as a named playlist
- Load saved playlists with one click — auto-plays
- Import any YouTube URL directly:
- Single video: `youtube.com/watch?v= ...`
- Radio mix: `youtube.com/watch?v= ... &list=RD ... &start_radio=1`
- Regular playlist: `youtube.com/playlist?list=PL ...`
- Playlists stored permanently in `chrome.storage` — survive browser restarts

Up Next

- Dedicated panel showing upcoming songs in the queue
- Pre-loaded from YouTube's Radio mix (similar songs) via yt-dlp
- Click any upcoming song to jump to it immediately

Lyrics

- One-click lyrics display for the currently playing song
- Powered by lyrics.ovh — free, no API key needed
- Auto-cleans song titles (strips "Official Video", "[HD]" etc.) for better matching
- Auto-fetches when you open the lyrics panel while a song plays

- Updates automatically when song changes

Keyboard Shortcuts

- **Space** — Play / Pause
- **→** — Next song
- **←** — Previous song
- **M** — Mute / Unmute

Windows Media Controls

- Shows **song name and artist** in the Windows system media overlay
- All overlay buttons work: Play, Pause, Next, Previous
- Seek forward/back (10 seconds) also supported

Auto-Start

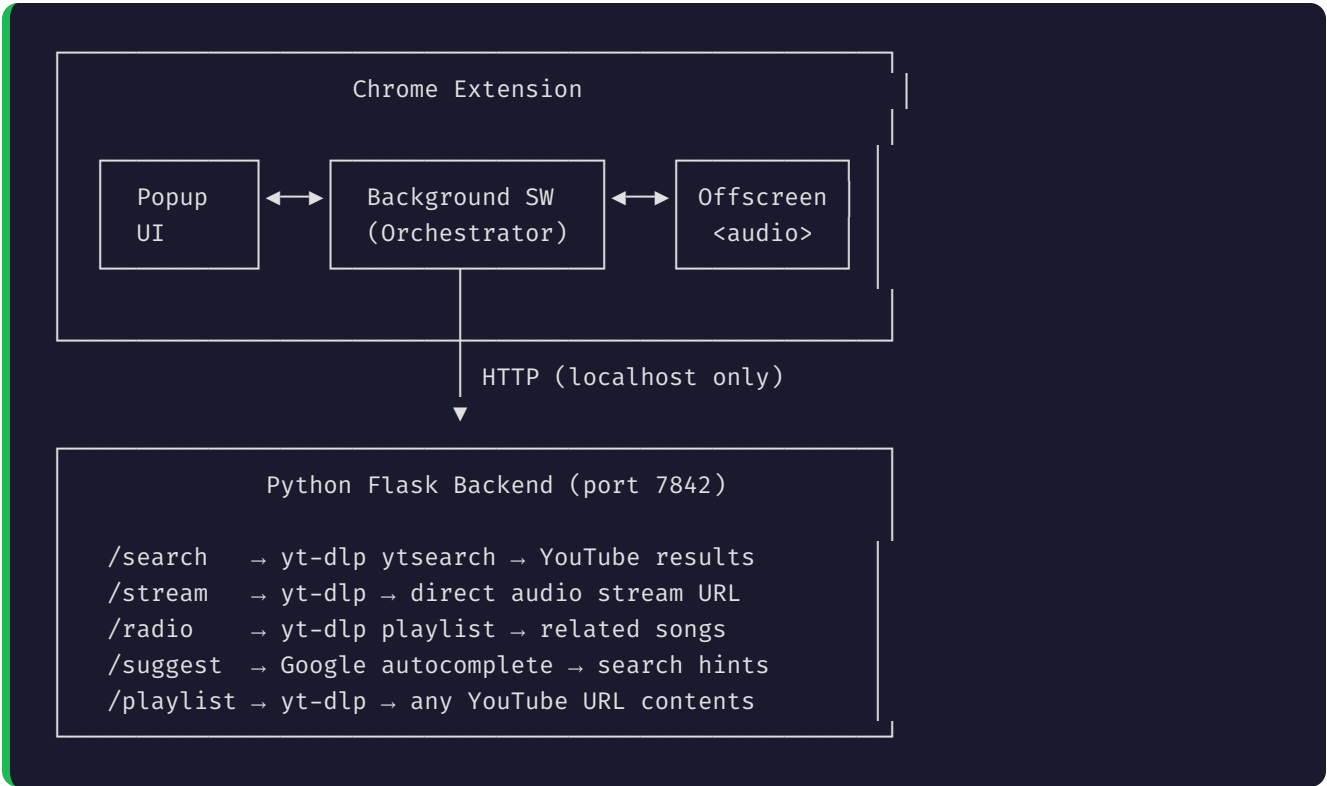
- Backend server placed in Windows Startup folder via **.vbs** script
- Runs **silently in the background** on every Windows login — no console window
- Extension has **auto-retry** — waits for backend to start before playing

State Persistence

- Song name, artist, thumbnail restored when popup is reopened
- Progress bar and time restored to exact position
- Play/pause state correctly shown on reopen
- Volume saved permanently in localStorage



Architecture



Key Design Decisions

Decision	Why
Offscreen document for audio	Service workers can't play audio; offscreen docs can
Local Python backend	Can't run yt-dlp inside a Chrome extension directly
yt-dlp for everything	YouTube Data API has 10,000 unit/day quota; yt-dlp has none
Audio-only stream	Saves 60-80% bandwidth vs full video
Startup folder auto-start	Task Scheduler needs admin rights; Startup folder doesn't
Stream URL caching	yt-dlp takes 2-5s per video; cache makes replays instant

 Tech Stack

Layer	Technology
Extension UI	HTML5, CSS3, JavaScript (Manifest V3)

Layer	Technology
Extension APIs	<code>chrome.offscreen</code> , <code>chrome.storage</code> , <code>chrome.runtime</code> , <code>chrome.scripting</code>
Audio	HTML5 <code><audio></code> element, Media Session API
Backend	Python 3, Flask, flask-cors
YouTube interaction	yt-dlp (no API key)
Lyrics	lyrics.ovh REST API
Suggestions	Google Suggest API (no key)
Auto-start	Windows VBScript + Startup folder



Setup

Requirements

- Windows 10 or 11
- Google Chrome (any recent version)
- Python 3.8 or higher

Step 1 — Install Python packages

```
pip install yt-dlp flask flask-cors
```

Step 2 — Start the backend (first time only)

```
python server/server.py
```

Or double-click `server/start_server.bat`

The backend will now **auto-start silently on every Windows login** — you never need to do this again after the first time.

Step 3 — Load the extension in Chrome

1. Open Chrome → go to `chrome://extensions`
2. Enable **Developer mode** (toggle in top right)
3. Click **Load unpacked**
4. Select the `music.sortcut` folder
5. Click the MusicFlow icon in your toolbar

That's it — search for any song and click play.

Usage

Basic playback

1. Type a song or artist in the search box
2. Click any result to play it
3. Use the player controls at the bottom

Import a YouTube playlist or radio mix

1. Click the  **Playlists** button in the header
2. Click **↓ URL**
3. Paste any YouTube URL (video, playlist, or radio mix link)
4. Press Enter — it loads and starts playing automatically

Save your queue as a playlist

1. Search for songs to build a queue
2. Click the  **Playlists** button
3. Click **+ Save Queue**
4. Give it a name — saved forever

See upcoming songs

1. Click the **Up Next** button (✓ icon) in the header
2. See the pre-loaded related songs
3. Click any to play it immediately

Get lyrics

- 1. Play any song
- 2. Click the  **Lyrics** button in the header
- 3. Lyrics appear automatically

Keyboard Shortcuts

Key	Action
 Space	Play / Pause
 Arrow	Next song
 Arrow	Previous song
 M	Mute / Unmute
 ↑ /  ↓ (in search)	Navigate suggestions
 Esc (in search)	Close suggestions
 Enter (in search)	Search

File Structure

music.sortcut/	
— manifest.json	Chrome extension config (Manifest V3)
— popup.html	Main UI layout
— popup.css	Spotify-dark theme styles
— popup.js	All player logic: - Search & suggestions - Playback controls - Playlist management - Lyrics fetching - State persistence - Keyboard shortcuts
— background.js	Service worker: - Manages offscreen document lifecycle - Fetches stream URLs from backend - Routes messages between components
— offscreen.html	Hidden audio player page
— offscreen.js	Audio element controller: - Plays stream URLs - Media Session API (Windows overlay) - Forwards media key events
— content.js	(Reference only)
— icons/	
— icon16.png	
— icon48.png	
— icon128.png	
— server/	
— server.py	Flask backend: - /search — yt-dlp YouTube search - /stream — audio URL extraction - /radio — YouTube Radio mix - /suggest — search autocomplete - /playlist — load any YouTube URL
— start_server.bat	Manual start (with console window)
— start_server_silent.vbs	Auto-start on Windows login (silent)

💡 Why No API Quota?

The YouTube Data API v3 gives 10,000 units/day free. Each search costs 100 units = only 100 searches/day. That runs out fast.

MusicFlow uses **zero YouTube Data API quota** because:

- **Search** → `yt-dlp ytsearch20:query` — scrapes YouTube search results directly
- **Stream URL** → `yt-dlp --get-url` — extracts the video's audio stream URL
- **Radio/Up Next** → `yt-dlp --flat-playlist` on the radio mix URL
- **Suggestions** → Google's public autocomplete endpoint (no key needed)

- **Lyrics** → lyrics.ovh (completely separate service, free)

The only remaining use of the YouTube Data API key in the code is as a fallback reference — it's never actually called during normal use.

License

MIT License — free to use, modify, and distribute.

Credits

- [yt-dlp](#) — the engine that makes everything possible
 - [lyrics.ovh](#) — free lyrics API
 - [Flask](#) — lightweight Python web framework
 - Inspired by Spotify's UI design language
-

Built with ❤️ by [Sumitboii](#)